

DOC API GitHub

ÍNDICE

ÍNDICE

Introdução

Definição e Contexto da GitHub REST API

Características Técnicas Fundamentais

Arquitetura da Integração

Componentes Principais da Arquitetura:

Sistema de Autenticação OAuth 2.0

Fundamentação Técnica da Escolha

Problemas Resolvidos pelo OAuth 2.0

Componentes do Fluxo OAuth 2.0

Escopos de Permissão

Fluxo de Autorização Detalhado

Considerações de Segurança do Fluxo

Implementação Técnica em Flutter

Catálogo de Rotas da API GitHub

Estrutura Organizacional dos Endpoints

Catálogo de Rotas da API GitHub

Estrutura Organizacional dos Endpoints

Rotas Principais Seleccionadas para o CodeRats

Rota de Perfil do Usuário Autenticado

Rota de Listagem de Repositórios

Rota de Listagem de Commits por Repositório

Fluxos de Busca de Commits - Estratégias e Implementação

Visão Geral das Abordagens de Busca

Introdução

Definição e Contexto da GitHub REST API

A GitHub REST API representa a interface principal para integração programática com a plataforma GitHub. Trata-se de uma API RESTful que expõe endpoints HTTP/HTTPS

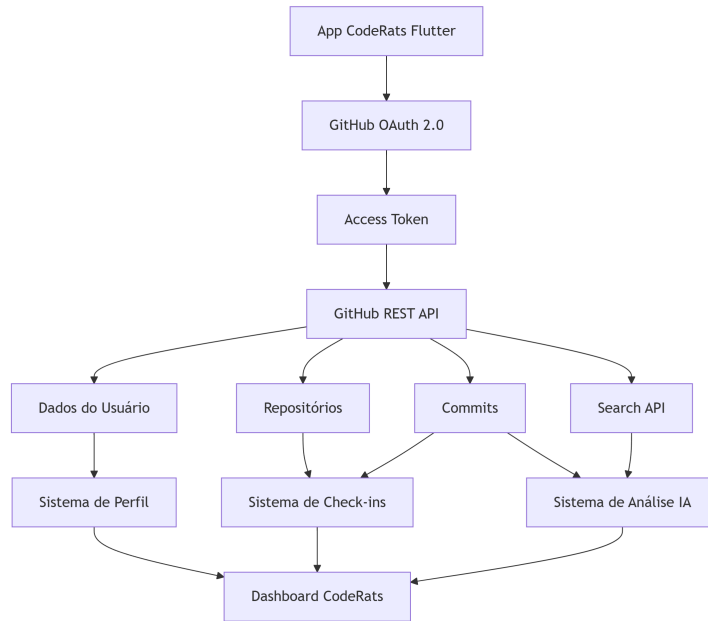
para acesso e manipulação de dados relacionados a repositórios, usuários, commits e outras entidades do ecossistema GitHub.

▼ Características Técnicas Fundamentais

- Protocolo de Comunicação: HTTP/HTTPS com métodos REST padrão (GET, POST, PUT, DELETE);
- Formato de Dados: JSON como formato exclusivo para requisições e respostas;
- Mecanismo de Autenticação: Suporte a OAuth 2.0, tokens de acesso pessoal e autenticação básica;
- Controle de Rate Limiting: 5.000 requisições por hora para aplicações autenticadas;
- Versionamento: API v3 (REST tradicional) e v4 (GraphQL) disponíveis;
- Paginação: Suporte nativo para paginação de resultados com cursor-based navigation.

Arquitetura da Integração

A integração entre o CodeRats e a GitHub API segue uma arquitetura em camadas que garante segurança, escalabilidade e manutenibilidade. A arquitetura proposta estabelece fluxos claros de dados e responsabilidades entre os diferentes componentes do sistema.



Fluxo Arquitetural Integração App-GitHub

▼ Componentes Principais da Arquitetura:

- **Camada de Autenticação:** Gerencia o fluxo OAuth 2.0 completo;
- **Camada de API Client:** Implementa chamadas específicas para endpoints GitHub;
- **Camada de Processamento:** Transforma e valida dados recebidos da API;
- **Camada de Análise IA:** Processa commits individuais para geração de relatórios;
- **Camada de Apresentação:** Interface Flutter para interação do usuário.

Sistema de Autenticação OAuth 2.0

Fundamentação Técnica da Escolha

▼ Problemas Resolvidos pelo OAuth 2.0

- **Delegação Segura:** Permite que usuários concedam acesso limitado sem compartilhar credenciais;

- **Escopos Granulares:** Controle preciso sobre permissões concedidas à aplicação;
- **Tokens Revogáveis:** Capacidade de revogação instantânea de acesso;
- **Padrão Industry:** Compatibilidade com práticas modernas de segurança.

▼ Componentes do Fluxo OAuth 2.0

- **Resource Owner:** Usuário final do GitHub
- **Client:** Aplicação CodeRats (confidencial)
- **Authorization Server:** Endpoints OAuth do GitHub
- **Resource Server:** API REST do GitHub
- **Authorization Grant:** Código de autorização temporário

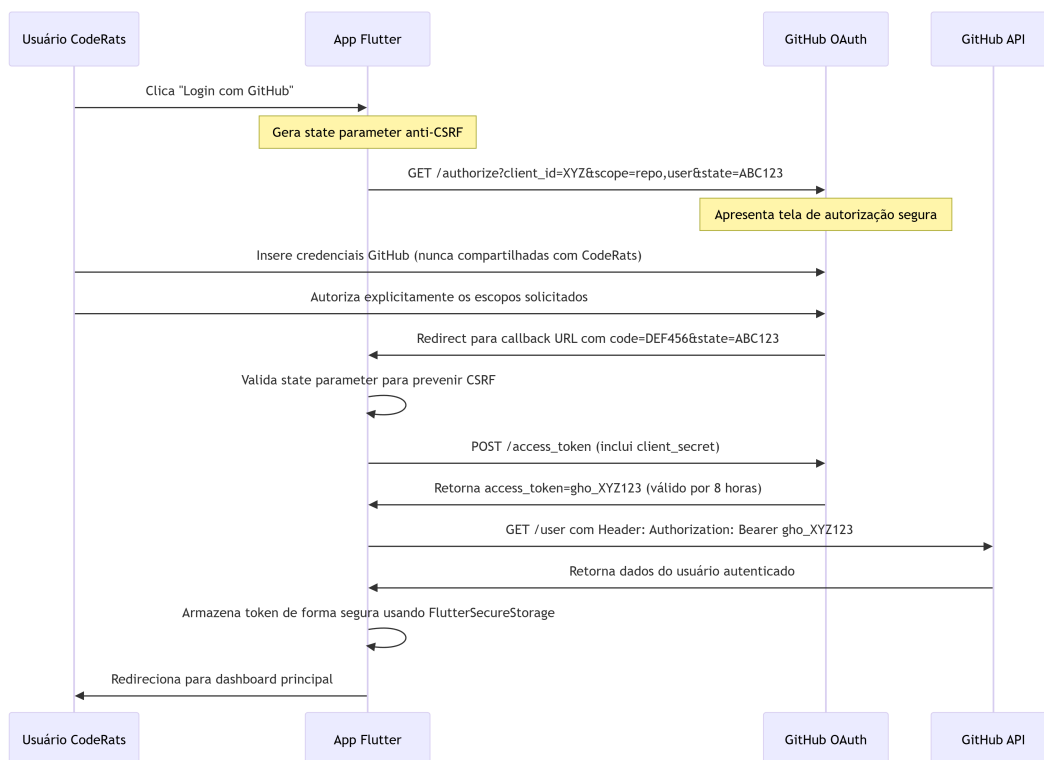
Escopos de Permissão

Os escopos definem os limites de acesso que o CodeRats solicita aos usuários:

```
class GitHubScopes {  
    // Acesso completo a repositórios públicos e privados  
    // Necessário para buscar commits em repositórios privados  
    static const String repo = 'repo';  
  
    // Acesso a dados do perfil do usuário  
    static const String user = 'user';  
}
```

Fluxo de Autorização Detalhado

O fluxo OAuth 2.0 implementado segue o **Authorization Code Grant**, considerado o mais seguro para aplicações móveis:



Definição técnica do Fluxo de Autorização

▼ Considerações de Segurança do Fluxo

- **State Parameter:** Prevenção contra ataques CSRF;
- **Client Secret:** Armazenado apenas no backend (não no app);
- **Token Storage:** Uso de FlutterSecureStorage para armazenamento criptografado;
- **Token Lifetime:** Tokens têm validade limitada (8 horas padrão).

Implementação Técnica em Flutter

A implementação do fluxo OAuth no CodeRats utiliza pacotes específicos do Flutter para garantir segurança e compatibilidade com as diretrizes de desenvolvimento móvel.

```
class GitHubOAuthConfig {
  static const String clientId = 'SEU_CLIENT_ID';
  static const String clientSecret = 'SEU_CLIENT_SECRET';
}
```

```
static const String redirectUrl = 'coderats://auth';
static const List < String > scopes = ['repo','user'];
static const String authUrl = 'https://github.com/login/oauth/authorize';
static const String tokenUrl = 'https://github.com/login/oauth/access_tok
en';
static const String apiBaseUrl = 'https://api.github.com';
}
```

Catálogo de Rotas da API GitHub

Estrutura Organizacional dos Endpoints

A GitHub API organiza seus endpoints de forma lógica por domínios de recurso. Para o CodeRats, focamos em três categorias principais:

1. **User Endpoints:** Operações relacionadas ao usuário autenticado
2. **Repository Endpoints:** Gestão e consulta de repositórios
3. **Commit Endpoints:** Acesso a histórico e detalhes de commits
4. **Search Endpoints:** Buscas avançadas através de múltiplos repositórios

Catálogo de Rotas da API GitHub

Estrutura Organizacional dos Endpoints

A GitHub API organiza seus endpoints de forma lógica por domínios de recurso. Para o CodeRats, focamos em três categorias principais:

1. **User Endpoints:** Operações relacionadas ao usuário autenticado
2. **Repository Endpoints:** Gestão e consulta de repositórios
3. **Commit Endpoints:** Acesso a histórico e detalhes de commits
4. **Search Endpoints:** Buscas avançadas através de múltiplos repositórios

Rotas Principais Selecionadas para o CodeRats

Rota de Perfil do Usuário Autenticado

Endpoint: `GET /user`

Esta rota serve como ponto de partida para construção do perfil do usuário dentro do aplicativo. Ela fornece informações básicas de identificação e métricas públicas que permitem personalizar a experiência do usuário.

Dados Retornados Relevantes

- Informações de identificação (ID, login, nome);
- Dados de contato (email, quando disponível);
- Métricas de atividade (número de repositórios, seguidores);
- URLs para avatares e perfis;
- Metadados temporais (data de criação, última atualização).

Documentação Oficial:

REST API endpoints for users - GitHub Docs
Use the REST API to get public and private information about authenticated users.
 <https://docs.github.com/en/rest/users/users?apiVersion=2022-11-28#get-the-authenticated-user>



Rota de Listagem de Repositórios

Endpoint: `GET /user/repos`

Esta rota possibilita a descoberta dos repositórios disponíveis para o usuário, servindo como base para a seleção durante o processo de check-in. Ela retorna tanto repositórios públicos quanto privados, desde que o usuário tenha concedido a permissão `repo`.

Parâmetros de Consulta Utilizados

- `type`: Filtro por tipo de relação (owner, member, all);
- `sort`: Critério de ordenação (updated, created, pushed, full_name);
- `direction`: Direção da ordenação (asc, desc);
- `per_page`: Número de resultados por página (1-100).

Dados Retornados Relevantes

- Identificação básica (nome, nome completo, ID);

- Metadados de visibilidade (público/privado);
- URLs de acesso e navegação;
- Estatísticas básicas (stars, forks, issues);
- Linguagem primária de programação;
- Datas de criação, atualização e último push.

Documentação Oficial:

REST API endpoints for repositories - GitHub Docs

Use the REST API to manage repositories on GitHub.

 <https://docs.github.com/en/rest/repos/repos?apiVersion=2022-11-28#list-repositories-for-the-authenticated-user>

 **GitHub Docs**

Rota de Listagem de Commits por Repositório

Endpoint: `GET /repos/{owner}/{repo}/commits`

Esta é a rota central para o funcionamento do sistema de check-ins. Ela permite recuperar o histórico de commits de um repositório específico, possibilitando ao usuário selecionar um commit individual para vincular a um check-in.

Parâmetros de Filtro Aplicados

- `author` : Filtro por autor específico (evita commits de outros colaboradores);
- `since` : Data de início para o período de consulta;
- `until` : Data de término para o período de consulta;
- `per_page` : Limite de commits retornados (10-20 para UI móvel).

Dados Retornados Relevantes

- SHA único do commit;
- Mensagem e descrição do commit;
- Metadados do autor (nome, email, data);
- Estatísticas de modificação (additions, deletions, files changed);
- URL para visualização no GitHub;

- Informações de parentesco (para merges).

REST API endpoints for commits - GitHub Docs

Use the REST API to interact with commits.

<http://docs.github.com/en/rest/commits/commits#list-commits>



Fluxos de Busca de Commits - Estratégias e Implementação

Visão Geral das Abordagens de Busca

O CodeRats emprega múltiplas estratégias para busca de commits, cada uma otimizada para cenários específicos de uso. A seleção da estratégia adequada balanceia fatores como performance, consumo de quota API, e experiência do usuário.

